The pandas package

Ezequiel López-Rubio

Department of Computer Languages and Computer Science

University of Málaga, Spain

Contents

- Introduction
- Series
- Data frames
- Input/output
- Conclusion

1. INTRODUCTION

1. Introduction

Overview

- The pandas package is a heavily used data analysis and manipulation tool for Python
- There are two main classes in pandas:
  - The Series class represents a one-dimensional labeled array that can hold any data type (integers, strings, floating point numbers, objects)
  - The DataFrame class represents a two-dimensional labeled data structure with columns of potentially different data types
- The usual way to import the package is:

```
import pandas as pd
```

2. SERIES

2. Series

Main features

- A Series is a one-dimensional numpy ndarray with a title (name) so that each array value has an associated label (index)
- A Series object is a valid argument to most numpy functions that process ndarrays
  - A key difference with ndarray is that operations between Series automatically align the data based on label
  - If a label is not found in one Series or the other, the result will be marked as missing NaN
- A Series is also like a fixed-size dict in that you can get and set values by index label

2. Series

Series creation

- A Series can be created from an ndarray:

```
>>> s = pd.Series(np.random.randn(5), index=["a", "b", "c", "d", "e"])
>>> s
a    0.469112
b   -0.282863
c   -1.509059
d   -1.135632
e    1.212112
dtype: float64
```

- The index must have the same length as the data ndarray

2. Series

Series creation

- A Series can be created from a dictionary:

```
>>> d = {"b": 1, "a": 0, "c": 2}
>>> s = pd.Series(d)
>>> s
b    1
a    0
c    2
dtype: int64
```

- If an index is passed to the Series constructor, then the missing values in the dictionary will be filled with NaNs

2. Series

Series creation

- A Series can be created from a single scalar value:

```
>>> s = pd.Series(5.0, index=["a", "b", "c", "d", "e"])
>>> s
a    5.0
b    5.0
c    5.0
d    5.0
e    5.0
dtype: float64
```

- The value is repeated to match the length of the index

2. Series

Using numpy functions

- Operations such as slicing will also slice the index

```
>>> s[0]
0.4691122999071863
>>> s[:3]
a    0.469112
b   -0.282863
c   -1.509059
dtype: float64
>>> s[s > s.median()]
a    0.469112
e    1.212112
dtype: float64
```

2. Series

Usage as a fixed size dictionary

- You can get and set values by index label

```
>>> s["a"]
0.4691122999071863
>>> s["e"] = 12.0
>>> s
a     0.469112
b    -0.282863
c    -1.509059
d    -1.135632
e    12.000000
dtype: float64
>>> "e" in s
True
```

2. Series

Label alignment

- Operations between Series automatically align the data based on label

```
>>> s[1:] + s[:-1]
a         NaN
b   -0.565727
c   -3.018117
d   -2.271265
e         NaN
dtype: float64
```

- The missing labels can be removed with the pd.Series.dropna function

2. Series

Naming

- The Series class has a name attribute

```
>>> s = pd.Series(np.random.randn(5), name="myname")
>>> s
0   -0.494929
1    1.071804
2    0.721555
3   -0.706771
4   -1.039575
Name: myname, dtype: float64
```

- A Series can be renamed with pd.Series.rename

2. Series

Useful operations

- Arithmetic, statistical, comparison and Boolean operations: like those of ndarrays
  - For operations between two Series, labels are aligned
- Sorting: by values (pd.Series.sort_values) or by index (pd.Series.sort_index). It can be done in place, i.e., modifying the original Series, or returning a new Series
- Number of unique values (pd.Series.nunique), number of occurrences of each unique value (pd.Series.value_counts)
- Position/label of the minimum/maximum value (pd.Series.argmin, pd.Series.idxmin, pd.Series.argmax, pd.Series.idxmax)
- Smallest/largest values (pd.Series.nsmallest, pd.Series.nlargest)

3. DATA FRAMES

3. Data frames

Main features

- The DataFrame class is the most used of the pandas package
- It is similar to a spreadsheet, a SQL table, or a dictionary of Series objects
- It is a two-dimensional structure where each column is a Series object, and each row is a sample
- Along with the data, you can optionally store row labels (index) and column labels (columns)

3. Data frames

Data frame creation

- A DataFrame can be created from a dictionary of Series or dictionaries
  - Dictionaries will first be converted to Series
- The resulting index will be the union of the indexes of the various Series

```
>>> d = { "one": pd.Series([1.0, 2.0, 3.0], index=["a", "b", "c"]),
   "two": pd.Series([1.0, 2.0, 3.0, 4.0], index=["a", "b", "c", "d"]) }
>>> pd.DataFrame(d)
 one  two
a  1.0  1.0
b  2.0  2.0
c  3.0  3.0
d  NaN  4.0
```

3. Data frames

Data frame creation

- A DataFrame can also be created from a dictionary of ndarrays or lists
  - The ndarrays must all be the same length
  - If an index is passed, it must also be the same length

```
>>> d = {"one": [1.0, 2.0, 3.0, 4.0], "two": [4.0, 3.0, 2.0, 1.0]}
>>> pd.DataFrame(d, index=["a", "b", "c", "d"])
 one  two
a  1.0  4.0
b  2.0  3.0
c  3.0  2.0
d  4.0  1.0
```

3. Data frames

Data frame creation

- A third method to create a DataFrame is from a list of dictionaries
  - Each dictionary represents a sample
  - The keys of the dictionaries are the column names

```
>>> d = [{"a": 1, "b": 2}, {"a": 5, "b": 10, "c": 20}]
>>> pd.DataFrame(d, index=["first", "second"])
        a   b     c
first   1   2   NaN
second  5  10  20.0
```

3. Data frames

Column management

- A DataFrame can be treated like a dictionary of Series objects

```
>>> df["one"]   # Accessing a column
a    1.0
b    2.0
c    3.0
d    NaN
Name: one, dtype: float64
>>> df["three"] = df["one"] * df["two"] # Insertion of a new column
>>> del df["two"] # Deletion of a column
>>> three = df.pop("three") # Popping a column
```

3. Data frames

Column management

- If a scalar value is inserted, it will be replicated to fill the column
- When inserting a Series that does not have the same index as the DataFrame, it will be conformed to the DataFrame's index

```
>>> df["foo"] = "bar"
>>> df["one_trunc"] = df["one"][:2]
>>> df
   one   flag  foo  one_trunc
a  1.0  False  bar        1.0
b  2.0  False  bar        2.0
c  3.0   True  bar        NaN
d  NaN  False  bar        NaN
```

- By default, columns get inserted at the end. DataFrame.insert inserts at a particular location in the columns

3. Data frames

Indexing and selection

Operation	Syntax	Result
Select column	df[col]	Series
Select row by label	df.loc[label]	Series
Select row by integer location	df.iloc[loc]	Series
Slice rows	df[5:10]	DataFrame
Select rows by Boolean vector	df[bool_vec]	DataFrame

3. Data frames

Label alignment

- Operations between DataFrames automatically align on both the columns and the index (row labels). The resulting object will have the union of the column and row labels

```
>>> df = pd.DataFrame(np.random.randn(6, 4), columns=["A", "B", "C", "D"])
>>> df2 = pd.DataFrame(np.random.randn(3, 3), columns=["A", "B", "C"])
>>> df + df2
          A         B         C   D
0  0.045691 -0.014138  1.380871 NaN
1 -0.494922 -1.649727 -1.084601 NaN
2 -1.047551 -0.748572 -0.805479 NaN
3       NaN       NaN       NaN NaN
4       NaN       NaN       NaN NaN
5       NaN       NaN       NaN NaN
```

3. Data frames

Label alignment

- For operations between a DataFrame and a Series, the default behavior is to align the Series index on the DataFrame columns, thus broadcasting row-wise:

```
>>> df - df.iloc[0]
          A         B         C         D
0  0.000000  0.000000  0.000000  0.000000
1 -1.359261 -0.248717 -0.453372 -1.754659
2  0.253128  0.829678  0.010026 -1.991234
3 -1.311128  0.054325 -1.724913 -1.620544
4  0.573025  1.500742 -0.676070  1.367331
5 -1.741248  0.781993 -1.241620 -2.053136
```

3. Data frames

Basic operations

- An overview of the DataFrame object can be obtained with pd.DataFrame.describe
- Arithmetic operations with scalars and Boolean operators operate element-wise
- To transpose, access the T attribute or pd.DataFrame.transpose()
- Most numpy functions can be called directly on Series and DataFrame objects
- Large DataFrame objects are truncated to be displayed on the console
  - The leading and trailing samples can be displayed with pd.DataFrame.head and pd.DataFrame.tail, respectively

3. Data frames

Basic operations

- Essential attributes of DataFrame objects:
  - pd.DataFrame.shape gives the axis dimensions of the object, consistent with ndarray
  - pd.DataFrame.index gets/sets the index (row labels)
  - pd.DataFrame.columns gets/sets the column labels
- Arithmetic functions have the option fill_value, i.e., a value to substitute when at most one of the values at a location are missing
- NaNs can be substituted with pd.DataFrame.fillna
- DataFrame objects can be combined with pd.Dataframe.combine
- Statistical operations take an axis argument: axis=0 (default) for the index, axis=1 for the columns
  - By default, NaNs are ignored (option skipna=True)

4. INPUT/OUTPUT

4. Input/output
Fundamentals

- The pandas package includes an input/output API consisting of two sets of functions, readers and writers
- The reader functions usually return a pandas object (Series or DataFrame)
- The writer functions convert a pandas object into the desired file type
- Most standard file types are supported. Each file type is managed by a different function

4. Input/output
Fundamentals

Data description	Reader	Writer
CSV	read_csv	to_csv
JSON	read_json	to_json
HTML	read_html	to_html
XML	read_xml	to_xml
MS Excel	read_excel	to_excel
Python pickle format	read_pickle	to_pickle
SQL	read_sql	to_sql

4. Input/output
Reading CSV files

- The pd.read_csv function is used to read a CSV file and obtain a DataFrame object
- The CSV file can be read from a file path, URL, open file or StringIO (filepath_or_buffer argument)
- The field delimiter must be specified with sep
- If the file has no header, then set header=None and specify the names argument. Otherwise, you should specify the header line number in the header argument
- Column datatypes can be specified with the dtype argument
- The thousands separator for integers should be specified with the thousands argument

4. Input/output

Reading Excel files

- The pd.read_excel function is used to read an Excel file and obtain a DataFrame object
  - Excel 2003 (.xls) files are read using the xlrd package
  - Newer (.xlsx) files are read using the openpyxl package
- The file path and the sheet name must be specified as arguments
- The pd.ExcelFile class can be used if multiple sheets must be imported
- The usecols argument is employed to specify a subset of columns
- The data type for a column can be specified with the dtype argument

5. CONCLUSION

5. Conclusion

- The pandas package is a very convenient tool to manage datasets for machine learning in Python
- Most standard dataset file formats are supported both for reading and writing
- Loops should be avoided at all costs in order to boost the performance of the code
- You must never modify something you are iterating over
- Missing and erroneous values should be managed by the pandas utilities before the data are presented to the machine learning algorithms